

Auditoria Arquitetural

OpenGenerative + Hermes / Showrunner

Leitura completa do repositório **showrunner-studio** para decidir onde encaixar o Agent Gateway e o runtime Hermes. O que existe hoje, o que não existe, o que pode ser reaproveitado — e a menor sequência de passos até “**crie uma imagem de um dragão**” aparecer inline no chat.

Escopo desta auditoria. Nenhum arquivo do repositório foi criado, alterado ou removido. Nenhuma migração, dependência ou refactor foi executado. O objetivo é compreender o terreno antes de qualquer implementação.

REPOSITÓRIO
/media/douglas/SSD2/dev/open/showrunner-studio
BASE ANALISADA
62 arquivos-fonte · 15.071 linhas · 21 arquivos de teste
STACK
Next.js 15.5.15 · React 19 · Tailwind v4 · JavaScript

DATA DA AUDITORIA
2 de setembro de 2026
CONTROLE DE VERSÃO
o diretório não é um repositório Git
RUNTIME EXTERNO
ComfyUI 127.0.0.1:8188 · FFmpeg · FFprobe

Sumário

01	Arquitetura atual	3
	Frontend, backend, jobs, assets, modelo de projeto e integração ComfyUI	
02	Diagrama da arquitetura atual	9
	Estado do sistema em 2 de setembro de 2026	
03	Componentes reaproveitáveis	12
	O que já está construído e deve ser mantido	
04	Componentes que ainda não existem	13
	Ausências e seu impacto sobre a integração	
05	Pontos de risco	14
	Oito riscos, do bloqueante ao acessório	
06	Onde encaixar o Agent Gateway	16
	Diretório, responsabilidades, interfaces e dependências	
07	Onde encaixar o domínio	18
	Project, Scene, Asset, Job, AgentThread, AgentMessage	
08	Arquivos que provavelmente mudarão	19
	Quinze arquivos, com risco estimado por item	
09	Arquivos que não deveriam ser alterados	20
	O que já está pago e não se recompra	
10	Plano incremental	21
	Onze passos testáveis, do controle de versão ao streaming	
11	Diagrama da arquitetura proposta	24
	Depois dos passos 1 a 8, com as camadas novas em destaque	
12	Recomendação final	26
	O que fazer primeiro e por quê	

As seções A a I solicitadas na investigação — frontend, backend, jobs, assets, modelo de projeto, ComfyUI, Agent Gateway, domínio mínimo e primeiro vertical slice — estão distribuídas entre os capítulos acima: A a F dentro do capítulo 1, G no capítulo 6, H no capítulo 7 e I no capítulo 10.

Arquitetura atual

ANTES DE TUDO — O NOME

Não existe projeto “OpenGenerative” neste repositório. O `package.json` declara `"name": "showrunner-studio"`. “Open Generative AI” aparece exclusivamente em `THIRD_PARTY_NOTICES.md` como atribuição MIT de *padrões* derivados (o `TABS` da navegação, o formato do catálogo de modelos, a disciplina do Prompt Composer) — e o próprio arquivo afirma: *“Nenhum arquivo do Open Generative AI foi copiado para este repositório.”*

Consequência prática para a decisão em pauta: o produto **já se chama Showrunner**. Batizar o agente de “Showrunner” cria colisão de nome dentro da própria plataforma (a aba `agente`, `components/screens/AgentScreen.jsx`, e o produto inteiro). Vale resolver isso antes de escrever a primeira linha.

A Frontend

Next.js 15.5.15 · React 19 · Tailwind v4 · JavaScript puro (sem TypeScript, sem ESLint configurado, sem biblioteca de estado). Três dependências de produção no total.

- **Rotas:** duas. `app/page.js` faz `redirect('/studio')` e `app/studio/[...slug]/page.js` é uma rota catch-all opcional que resolve o slug contra `TABS` e renderiza `components/StudioShell.jsx`. Toda “navegação” interna é troca de componente em memória mais `router.push()` — não há page por área.
- **Sidebar:** gerada de `components/navigation.js` — lista declarativa `TABS` com 13 entradas agrupadas em *Produção, Estúdios, Montagem, Automação, Sistema*. `NAV_GROUPS` é derivado dela.
- **Onde entra a aba “Agent”:** já existe uma — `{ id: 'agente', group: 'Automação' }` em `components/navigation.js:18`, mapeada em `SCREENS.agente` (`components/StudioShell.jsx:36`) para `components/screens/AgentScreen.jsx`. Não é preciso criar rota: basta substituir o conteúdo dessa tela (ou acrescentar uma segunda entrada em `TABS`, o que é uma linha).
- **Chat existente:** sim, completo visualmente e inteiramente falso. `AgentScreen.jsx` tem bolhas usuário/agente, avatares, sugestões clicáveis, auto-scroll, *“o agente está formulando...”*, textarea com Enter-envia, e **anexos inline de imagem e vídeo dentro da mensagem** (linhas 297–318). O cérebro é `lib/agentScript.js`: uma máquina de estados de 6 perguntas com `setTimeout(700)`. Nenhuma LLM, nenhuma rede.
- **Componentes de mídia:** `components/ui/MediaFrame.jsx` expõe `ImageFrame`, `VideoFrame` e `MediaPreview`. `VideoFrame` ramifica: se `item.real && item.mediaUrl`, delega a `components/ui/RealVideoPlayer.jsx` (player nativo com retry e cache-bust); senão percorre quadros SVG simulados. *Isto é exatamente o gancho de renderização inline que o agente precisa.*
- **Upload:** dois caminhos distintos. Referências decorativas viram data-URL por `FileReader` e nunca saem do navegador (`ImageScreen.jsx:58`). O upload real

preserva o objeto `File` e o envia como `multipart/form-data` a `/api/comfy/generate` via `submitVideoWithFrames()` (`VideoScreen.jsx:292`).

- **Streaming:** **NÃO EXISTE** Nenhum `EventSource`, nenhum SSE, nenhum WebSocket no cliente. O único WebSocket do projeto é servidor↔ComfyUI em `lib/server/comfy/events.js`. O acompanhamento é **polling de 1,5 s** em `CinemaScreen` / `VideoScreen` e de intervalo próprio em `LogsScreen`.
- **Estado:** um único React Context — `components/StudioContext.jsx` — com 12 fatias persistidas em `localStorage` sob o prefixo `showrunner.` (`lib/storage.js`). A hidratação ocorre pós-montagem; `ready` guarda o primeiro render.
- **Como um output aparece:** a tela submete → recebe `jobId` → faz polling → ao ver `result.url` chama `registrarResultado()`, que empurra um objeto `{kind: 'video', real: true, mediaUrl: '/api/media/video/...'}` em `generations` → o React re-renderiza e `VideoFrame` monta o `RealVideoPlayer`. O `<video>` aponta para `app/api/media/[...segments]/route.js`, que serve o MP4 com suporte a `Range`.

B Backend

Não existe backend separado. O backend é o Next.js: 13 Route Handlers em `app/api/`, todos `runtime = 'nodejs'` e `dynamic = 'force-dynamic'`. Sem Express, sem NestJS, sem controllers, sem middleware, sem autenticação, sem ORM, **sem banco de dados**.

ROTA	VERBO	DELEGA PARA	PAPEL
<code>api/comfy/generate</code>	POST	<code>submitGeneration()</code>	Aceita JSON (t2v) ou multipart (i2v/ftf). Valida e devolve 202.
<code>api/comfy/status</code>	GET	<code>pollJob()</code>	Consulta <code>/history</code> e <code>/queue</code> do ComfyUI.
<code>api/comfy/result</code>	POST	<code>finalizeJob()</code>	Baixa, valida com <code>ffprobe</code> e publica. Idempotente.
<code>api/comfy/cancel</code>	POST	<code>cancelJob()</code>	Remove da fila ou interrompe.
<code>api/comfy/jobs</code>	GET	<code>listJobs()</code> , <code>recoverFromHistory()</code>	Lista e adota resultados órfãos.
<code>api/comfy/test</code>	POST/GET	<code>testConnection()</code>	5 checagens reais contra o ComfyUI.
<code>api/export/*</code>	4 rotas	<code>lib/server/export/runner.js</code>	Montagem FFmpeg: <code>start</code> · <code>status</code> · <code>cancel</code> · <code>list</code> .
<code>api/logs</code>	GET	<code>logStore().query()</code>	Leitura incremental por cursor.
<code>api/filmstrip</code>	GET	<code>getFilmstrip()</code>	4 quadros por clipe, cache por hash.
<code>api/media/[...segments]</code>	GET	<code>resolve*Path()</code>	Serve MP4/JPG. Aceita video, export, frame.

Padrão dos handlers: todos são finos e idênticos em forma — validam o corpo com uma regex (`/^[A-Za-z0-9_-]{1,64}$/`), chamam uma função de `lib/server/`, mapeiam o tipo do erro para um HTTP status (`WorkflowError` → 422, `ComfyError` → 502, `UploadError` → 400) e devolvem JSON. **Toda a lógica vive em `lib/server/`** — o que é excelente notícia para encaixar o Gateway.

Erros: quatro classes tipadas — `ComfyError`, `WorkflowError`, `UploadError`, `AssetResolutionError`, `PathValidationError`, `ExportArgsError`, `FfmpegError` — mais um sistema de diagnóstico completo em `lib/server/logs/`: 16 estágios nomeados, buffer circular de 500 eventos que protege os 50 erros mais recentes da rotação, persistência NDJSON com retenção de 7 dias, sanitização de caminhos, e correlação por `AsyncLocalStorage` — o que faz cada linha de log pertencer ao job certo mesmo com gerações simultâneas.

C Sistema de jobs

CAPACIDADE	ESTADO	ONDE
Abstração de job	EXISTE	<code>lib/server/comfy/jobs.js</code> — <code>createJob/getJob/updateJob/allJobs</code>
Estados nomeados	EXISTE	<code>comfy/status.js:7</code> — 9 estados, <code>TERMINAL_STATES</code> , <code>isTerminal()</code>
Progresso 0–1	EXISTE	<code>normalizeProgress()</code> + eventos progress do WebSocket
Cancelamento	EXISTE	<code>cancelJob()</code> (fila/interrupt) e <code>cancelExport()</code> (<code>SIGTERM</code> → <code>SIGKILL</code>)
Polling	EXISTE	Cliente 1,5 s; servidor lê <code>/history</code> então <code>/queue</code>
Processamento em background	EXISTE	<code>startExport()</code> dispara <code>executar()</code> sem <code>await</code> (<code>export/runner.js:151</code>)
Recuperação pós-restart	PARCIAL	<code>recoverFromHistory()</code> reconstrói do <code>/history</code> do ComfyUI — só vídeo, só com prefixo <code>video/showrunner</code>
Fila própria	NÃO EXISTE	A fila é a do ComfyUI. Sem limite de concorrência, sem prioridade, sem backpressure
Retry de geração	NÃO EXISTE	Só há retry de <i>download</i> (3×) em <code>baixarValidarPublicar()</code>
Persistência do job	NÃO EXISTE	Map em <code>globalThis[Symbol.for('showrunner.comfy.jobs')]</code> ; poda em 6 h
SSE / WebSocket ao cliente	NÃO EXISTE	—

D Assets e armazenamento

Há **dois mundos desconectados**, e essa fratura é o achado central desta auditoria.

- **Servidor:** arquivos em `runtime/projects/<projectId>/{videos,exports,cache/filmstrip}/`. Não há índice — o “registro” é o resultado de um `readdir (listRegisteredVideos())` em `export/assets.js:30`). O `jobId` é o nome do arquivo. Nenhuma metadata em disco.
- **Navegador:** o array `generations[]` no `localStorage` carrega toda a metadata que importa — `prompt`, `seed`, `aspect`, `duração`, `status` (`pendente/aprovado/revisão`), `revisionNote`, `jobId`, `mediaUrl`.

Existe abstração de Asset reutilizável? **NÃO** O mais próximo é a *allowlist* de `lib/server/export/assets.js` — bem construída, mas é resolução de segurança (identificador → caminho), não um modelo de domínio. Não tem tipo, não tem dono, não tem criação, não tem metadata, e é **exclusivamente** `.mp4`: `validateVideoFilename()` (`comfy/storage.js:31`) rejeita qualquer outra extensão por regex.

Imagens geradas: **NÃO EXISTEM** Nenhuma. Toda “imagem” do sistema é um SVG determinístico desenhado em tempo de execução por `lib/placeholder.js` a partir da seed. Uploads do usuário nunca são gravados localmente — vão em memória direto para `input/showrunner/` do ComfyUI.

E **Modelo de projeto**

CONCEITO	ESTADO	ONDE VIVE HOJE
Project	SÓ NO CLIENTE	<code>localStorage 'showrunner.projects'</code> ; forma em <code>demoProjects()</code> . No servidor é apenas um segmento de caminho validado por regex.
Scene	SÓ NO CLIENTE	<code>createScene()</code> em <code>lib/storyboard.js:21</code> — <code>{id,number,title,description,duration,modelId,status,image,videoId}</code> com 5 estados.
Asset	FRATURADO	Metadata no <code>localStorage (generations[])</code> ; bytes no disco do servidor. Nada os une exceto o <code>jobId</code> .
Job	SERVIDOR	Dois registros paralelos e independentes: <code>comfy/jobs.js</code> e <code>export/runner.js</code> .
Conversation	SÓ NO CLIENTE	<code>conversation.messages[]</code> — <code>{id,role,text,createdAt,attachments:[genId]}</code> .
Message	SÓ NO CLIENTE	Idem. Já suporta anexos por referência de id — a forma certa.
Take / Version	NÃO EXISTE	Regerar cria um <code>generation</code> novo e não relacionado. Não há linhagem.
Shot	NÃO EXISTE	—
Production	NÃO EXISTE	Sinônimo informal de <code>Project</code> na UI.
Approval	SÓ NO CLIENTE	Regra pura em <code>lib/approval.js</code> e <code>canEnterTimeline()</code> . O <i>estado</i> vive no navegador — e <code>export/runner.js:77</code> documenta que confia no cliente.

CONCEITO	ESTADO	ONDE VIVE HOJE
Memória do projeto	NÃO EXISTE	—

F Integração ComfyUI

A cadeia real, arquivo por arquivo:

CAMINHO DE UMA GERAÇÃO	
CinemaScreen.jsx / VideoScreen.jsx	
└─ lib/providers/ComfyUIProvider.js	submitVideo() · pollJob() · finalizeJob() · cancelJob()
└─ app/api/comfy/generate/route.js	valida · lê multipart · jobId nasce aqui
└─ lib/server/comfy/provider.js	ORQUESTRADOR (862 linhas)
└─ workflow.js	loadWorkflowTemplate() → validateWorkflow() → patchWorkflow()
└─ images.js	detectImageType() por magic bytes → safeFrameName()
└─ client.js	POST /upload/image → GET /view (confirma) → POST /prompt
└─ jobs.js	createJob() → updateJob(promptId, SUBMITTED)
└─ events.js	WebSocket /ws?clientId= – progresso fino, best-effort
└─ status.js	interpretHistory() · findInQueue() · findVideoOutput()
└─ storage.js	videoTempPath() → ffprobe → publishVideoFile() [rename atômico]

PEÇA	ARQUIVO · LINHA	REAPROVEITÁVEL PARA OG.GENERATE_IMAGE ?
Cliente HTTP	comfy/client.js	100% Genérico. submitPrompt, history, queue, view, uploadImage, interrupt.
Registro de jobs	comfy/jobs.js	100% Não sabe o que é vídeo.
Máquina de estados	comfy/status.js:7-63	100% STATES, findInQueue, interpretHistory são agnósticos.
Localizar saída	status.js:104	ADAPTAR findVideoOutput() filtra por isVideoFile() (mp4/webm/mkv/mov/m4v). Precisa de irmão para png/jpg/webp.
jobId ← nome do arquivo	status.js:138	ADAPTAR Regex exige extensão de vídeo.
Upload de entrada	comfy/images.js	100% Magic bytes, nome seguro derivado do jobId, papéis fechados. Modelo a seguir.
Progresso por WebSocket	comfy/events.js	ADAPTAR phaseForNode() tem ids do MiniMax hardcoded ('105:10', '105:14').
Armazenamento	comfy/storage.js	ESTENDER Padrão de segurança impecável (regex + assertInside + rename atômico), mas todo .mp4.
Carga do workflow	comfy/workflow.js	REESCREVER Ver abaixo.
Config	comfy/config.js	REESCREVER Ver abaixo.

O GARGALO REAL DO GENERATE_IMAGE

Existe um **único workflow**, com caminho absoluto codificado:

```
/media/douglas/SSD2/comfyui_data/workflows/minimax_h3_t2v_api.json  
(comfy/config.js:11). E validateWorkflow() recusa submeter qualquer grafo  
que não tenha exatamente os 11 ids de nó de NODE_IDS, com os class_type de  
NODE_CLASSES, apontando para os 4 .safetensors de REQUIRED_MODEL_FILES.  
Isso é uma trava deliberada e boa — mas significa que o carregamento de  
workflow precisa virar um registry antes que qualquer segundo modelo exista.
```

Ideogram: **ZERO CÓDIGO** Aparece como entrada de catálogo (`lib/models.js:21`, `providerId: 'local'`, `status: 'available'`) e como `modelId` em dados de demonstração. O `LocalProvider` lança `NotImplementedError` em `generateImage()`. Não há cliente, endpoint, chave nem workflow.

G-I Resumo do que a integração enfrenta

O sistema é maduro num eixo e inexistente no outro. O eixo *vertical* — submeter ao ComfyUI, acompanhar, validar, publicar, servir, exportar, diagnosticar — está construído com rigor incomum: rename atômico, validação por ffprobe antes de publicar, recuperação de jobs órfãos, allowlist contra path traversal, magic bytes contra extensão mentirosa, 21 arquivos de teste. O eixo *horizontal* — quem é o projeto, o que é uma cena, o que foi aprovado, o que aconteceu na conversa — **não existe no servidor**. Vive no `localStorage` de um navegador.

Diagrama da arquitetura atual

NAVEGADOR · única fonte de verdade do domínio

app/studio/[...slug]/page.js → StudioShell.jsx

- └ navigation.js TABS[13] ("agente" já existe, grupo Automação)
- └ StudioContext.jsx → localStorage "showrunner.*"
 - projects · generations · storyboard · timeline · characters
 - conversation · settings · cinema · cinemaJob · exportJob
 - activeProject · timelineRevision
- └ screens/ Home Create Image Video Cinema Characters Storyboard
 - Projects Library Workflows Agent Settings Logs
- └ providers/registry.js
 - ComfyUIProvider → fetch /api/comfy/* [ÚNICO provider real]
 - LocalProvider · ApiProvider → NotImplementedError
 - MockProvider → SVG determinístico

HTTP · polling 1,5 s
(sem SSE · sem WebSocket · sem auth)

NEXT ROUTE HANDLERS app/api/* runtime='nodejs' dynamic=force

comfy/ generate status result cancel jobs test
export/ start status cancel list
logs filmstrip media/[...segments]

Todos finos: regex → função de lib/server → mapeia erro para HTTP

lib/server/

comfy/ provider.js ORQUESTRADOR submit→poll→finalize→recover
 client.js workflow.js status.js images.js config.js
 jobs.js storage.js events.js
export/ runner.js args.js assets.js ffmpeg.js
media/ filmstrip.js
logs/ logger store persist context(AsyncLocalStorage) stages

ESTADO: Map em globalThis[Symbol.for('showrunner.comfy.jobs')]
Map em globalThis[Symbol.for('showrunner.export.jobs')]
→ sem fila · sem banco · perdido no restart · poda em 6 h

HTTP 127.0.0.1:8188

spawn(shell:false)

ComfyUI
POST /prompt
GET /queue /history
GET /view (Range)
POST /upload/image
WS /ws?clientId=

1 workflow, 11 nós fixos
minimax_h3_t2v_api.json

/usr/bin/ffmpeg /usr/bin/ffprobe

runtime/projects/<projectId>/
 videos/<jobId>.mp4
 exports/<exportId>.mp4
 cache/filmstrip/<hash>__<n>.jpg
runtime/logs/comfy-YYYY-MM-DD.ndjson

x nenhum banco de dados
x nenhuma autenticação

x nenhuma LLM
x nenhuma geração de imagem

Componentes reaproveitáveis

Aproveite estes tal como estão — são o ativo real do repositório.

COMPONENTE	ARQUIVO	POR QUÊ
Cliente ComfyUI	<code>lib/server/comfy/client.js</code>	Timeouts, AbortController, sanitização, recordExchange em toda troca. Serve qualquer workflow.
Registro de jobs	<code>lib/server/comfy/jobs.js</code>	73 linhas, agnóstico de mídia. Base direta do <code>og.get_job</code> .
Estados e leitura do ComfyUI	<code>lib/server/comfy/status.js</code>	Funções puras, sem I/O, cobertas por teste. Só <code>findVideoOutput</code> precisa de irmão.
Validação de imagem	<code>lib/server/comfy/images.js</code>	Magic bytes, nome derivado do jobId, papéis fechados. É o gabarito para todo upload futuro.
Segurança de caminho	<code>lib/server/comfy/storage.js</code>	<code>validateSegment</code> · <code>assertInside</code> · <code>rename</code> atômico · nunca sobrescreve. Estender, não substituir.
Allowlist de assets	<code>lib/server/export/assets.js</code>	Identificador → caminho, nunca caminho do cliente. Regra que o Gateway deve herdar.
Diagnóstico completo	<code>lib/server/logs/*</code>	Estágios, buffer com retenção de erro, NDJSON, correlação por ALS. Acrescentar CHANNELS. AGENT é uma linha.
Serviço de mídia	<code>app/api/media/[...segments]/route.js</code>	Range requests, nosniff, contenção de caminho. Acrescentar o tipo <code>image</code> é uma entrada no mapa TIPOS.
Player real	<code>components/ui/RealVideoPlayer.jsx</code>	Retry, cache-bust, mensagens de <code>MediaError</code> . Já é o vídeo inline.
Ramificação real/simulado	<code>components/ui/MediaFrame.jsx:102</code>	<code>item.real && item.mediaUrl</code> → player nativo. É o ponto exato de renderização inline no chat.
Chat com anexos	<code>components/screens/AgentScreen.jsx:272</code>	<code><Message></code> já resolve <code>attachments: [id]</code> contra <code>generations</code> e monta <code>ImageFrame/VideoFrame</code> . A UI do vertical slice já está pronta.
Loop de acompanhamento	<code>components/screens/CinemaScreen.jsx:126</code>	<code>acompanhar()</code> com <code>backoff</code> e 5 tentativas. Copiar o padrão, não o código.
Registry de providers	<code>lib/providers/registry.js</code>	Prova que a equipe já pensa em portas e adaptadores. O Gateway segue a mesma forma.
Suíte de testes	<code>tests/*.test.mjs</code>	21 arquivos com <code>node:test</code> , sem dependências. Estender é trivial.

Componentes que ainda não existem

AUSENTE	IMPACTO NA INTEGRAÇÃO
Persistência no servidor — qualquer banco	Bloqueante. O agente roda no servidor e hoje não consegue ler nem escrever projeto, cena, aprovação ou histórico.
Geração de imagem — qualquer caminho	Bloqueante para o slice 1. Nenhum workflow t2i, nenhum cliente Ideogram, nenhum armazenamento de PNG.
Regístry de workflows	Bloqueante. Um caminho absoluto e 11 ids fixos em comfy/config.js.
Camada de tools — catálogo, schema, dispatcher	Bloqueante. Não há nada análogo a og.*.
Streaming ao cliente — SSE ou WS	Alto. Sem isso, resposta de agente token-a-token é impossível; sobra polling.
Qualquer LLM	Alto. lib/agentScript.js é setTimeout sobre um array de 6 perguntas.
Autenticação / autorização	Alto assim que o Gateway existir — as rotas hoje são abertas por design (“sem login, sem chave”).
Take / Version / linhagem	Médio. “Anime essa imagem” exige relação asset→asset, que não existe.
Storage de imagem	Médio. validateVideoFilename é mp4-only; a rota de mídia não tem tipo image.
Aprovação no servidor	Médio. og.request_approval não tem onde gravar nada.
Idempotência / limite de concorrência	Médio. Um agente em laço pode enfileirar dezenas de jobs sem freio.
TypeScript ou schema runtime	Baixo, mas relevante: argumentos de tool vindos de LLM precisam de validação declarada, e hoje ela é if à mão.

Pontos de risco

R1 • O domínio mora no navegador CRÍTICO

`projects`, `scenes`, `generations`, `conversation` e o estado de aprovação vivem em `localStorage` (`lib/storage.js:5`). O princípio nº 4 da decisão — “OpenGenerative é dono de projetos, cenas, assets, aprovações, memória” — **é falso hoje**, e Hermes não tem como cumprir `og.create_scene`, `og.update_scene` ou `og.request_approval` contra um estado que ele não enxerga. Nenhum outro risco importa antes deste.

R2 • `/interrupt` do ComfyUI é global CRÍTICO

`comfy/client.js:106` comenta: “o ComfyUI não aceita alvo específico”. `cancelJob(A)` com A em execução interrompe *o que estiver rodando*. Com um usuário sequencial isso é aceitável; com um agente disparando jobs paralelos, cancelar um mata outro. Sem fila própria com concorrência 1, isto vira corrupção silenciosa.

R3 • Jobs somem no restart ALTO

`Map` em `globalThis`, poda de terminais em 6 h (`comfy/jobs.js:66`). `recoverFromHistory()` mitiga, mas só para vídeo com prefixo `video/showrunner` e nome que volte a um `jobId` válido. Uma thread de agente que referencie `job_abc` depois de um reload de módulo do Next recebe “Job desconhecido”.

R4 • O workflow é um caminho absoluto de uma máquina ALTO

`comfy/config.js:11` aponta para `/media/douglas/SSD2/comfyui_data/...`. O projeto não é portátil, e `validateWorkflow()` rejeita qualquer outro grafo. Todo modelo novo — incluindo o de imagem — esbarra aqui.

R5 • Nenhuma autenticação ALTO

É uma decisão de produto explícita e correta hoje (“sem conta, sem chave”). Mas `/api/agent/*` aceitando texto livre que vira execução de tool é outra classe de superfície. Se o Gateway ouvir em algo além de `127.0.0.1`, precisa de token.

R6 • Aprovação é um portão de fluxo, não de segurança MÉDIO

`export/runner.js:77` declara isso textualmente. O cliente envia `approved:true` e o servidor acredita. Se o agente puder chamar `og.request_approval`, ele pode aprovar em nome do usuário — a menos que o estado de aprovação passe a viver no servidor (o que R1 já exige).

R7 • Sem backpressure MÉDIO

Nada limita submissões simultâneas. Um laço de agente com erro enche a fila da GPU. Não há `maxConcurrent`, timeout de job, nem orçamento por thread.

R8 • Dívidas menores BAIXO

- `README.md` está desatualizado — descreve “Fase 1, nenhum modelo é executado”, mas Cinema e Vídeo geram de verdade.
- `SCENE_REGISTRY` (`lib/timelineAssets.js:15`) contém dois `jobId` reais codificados — dado de uma máquina dentro do código.
- Sem TypeScript e sem validação de schema: argumentos de tool vindos de LLM chegam a `if`s manuais.
- Não é um repositório Git — não há rede de segurança para reverter.
- Toda a base é em português (identificadores, mensagens, comentários). O contrato de tools com Hermes precisará decidir o idioma e ser consistente.

Onde encaixar o Agent Gateway

A convenção do repositório é clara e deve ser seguida à risca: **rota fina** em `app/api/`, **lógica** em `lib/server/`, **adaptador de navegador** em `lib/providers/`. O Gateway é um irmão de `lib/server/comfy/` e `lib/server/export/` — nem acima, nem dentro.

Diretório

PROPOSTA — NADA IMPLEMENTADO

```
lib/server/agent/
  gateway.js                fachada: startTurn · continueTurn · getThread · cancelTurn
  AgentRuntimePort.js       o CONTRATO (classe base + erros), espelhando BaseProvider.js
  adapters/
    HermesRuntimeAdapter.js fala HTTP com o processo Hermes
    EchoRuntimeAdapter.js   determinístico, sem rede — é o que os testes usam
  tools/
    registry.js             catálogo og.* : nome, descrição, schema, handler
    schema.js              validação dos argumentos (o LLM erra; falhe cedo e legível)
    handlers/
      generateImage.js      → lib/server/generation/
      generateVideo.js      → lib/server/generation/
      getJob.js             → lib/server/comfy/jobs.js
      createScene.js        → lib/server/domain/scenes.js
      updateScene.js
      requestApproval.js
  threads.js                persistência de AgentThread / AgentMessage
  events.js                 fila de eventos do turno, consumida por SSE

app/api/agent/
  threads/route.js          GET lista · POST cria
  threads/[threadId]/route.js GET uma thread com mensagens
  messages/route.js         POST envia mensagem do usuário → dispara o turno
  events/route.js           GET SSE (ou polling por cursor, como /api/logs)
  cancel/route.js           POST interrompe o turno

lib/providers/AgentProvider.js  lado navegador, espelhando ComfyUIProvider.js
```

Responsabilidades

- **Possui o turno.** Recebe a mensagem, persiste, chama o runtime, executa as tools pedidas, devolve o resultado, persiste tudo de novo. Hermes nunca escreve no domínio direto.
- **Traduz.** Fronteira única entre o vocabulário do runtime (tool calls) e o vocabulário do Showrunner (jobs, assets, cenas).
- **Autoriza.** Só as tools do `registry.js` existem. Argumento fora do schema é erro, não tentativa.
- **Limita.** Concorrência, orçamento de tools por turno, timeout. É o único lugar onde isso pode viver.

- **Registra.** Emite em `lib/server/logs/` com um `CHANNELS.AGENT` novo, para que o agente apareça na aba Logs como o ComfyUI aparece.

Interfaces

Duas portas, ambas modeladas em `lib/providers/BaseProvider.js` — que já é exatamente esta ideia:

- `AgentRuntimePort` — `isAvailable()`, `unavailableReason()`, `testConnection()`, `run({thread, messages, tools, signal}) → AsyncIterable` de eventos `{type: 'text' | 'tool_call' | 'done' | 'error'}`. Trocar Hermes por outro runtime = escrever um segundo adaptador. O princípio nº 9 é atendido aqui.
- `ToolHandler` — `{name, description, schema, async execute(args, ctx)}`, onde `ctx` carrega `{threadId, projectId, logger}` e *nunca* o objeto de requisição.

Dependências permitidas

- `lib/server/domain/*` — projetos, cenas, assets, threads
- `lib/server/generation/*` — a camada de geração extraída de `comfy/`
- `lib/server/logs/*`
- Primitivas do Node

Dependências proibidas

- **Nada de** `components/`, `React` ou `lib/storage.js`. O Gateway roda no servidor; `localStorage` não existe para ele.
- **Nada de** `lib/server/comfy/client.js` **direto**. Se um handler falar HTTP com o ComfyUI, a regra “ComfyUI é controlado pelo Showrunner, não pelo Hermes” já foi violada no primeiro dia. Handlers chamam *generation*, que chama *comfy*.
- **Nada de** `next/server` **dentro de** `lib/server/agent/`. As rotas importam o gateway; o gateway não conhece HTTP — é o que o mantém testável com `node:test` como o resto.
- **Nenhum import do sentido inverso:** `comfy/` e `export/` nunca importam `agent/`.
- **Nenhuma chave de API no cliente.** A credencial do Hermes fica em variável de ambiente lida só no servidor, como `COMFY_URL`.

Onde encaixar o domínio

Novo diretório `lib/server/domain/`, irmão do Gateway. Ele passa a ser a autoridade; o `localStorage` vira *cache* de leitura, não fonte de verdade.

Persistência — a escolha

SQLite via `node:sqlite` (nativo a partir do Node 22; o projeto roda em Node 24 e o README exige ≥ 18). Zero dependências novas — o que preserva a virtude do repositório — e dá transação e consulta reais. O banco em `runtime/showrunner.db`, irmão de `runtime/projects/` e `runtime/logs/`, que já seguem esse padrão. Alternativa aceitável para o primeiro slice: um JSON por entidade sob `runtime/domain/`, migrando depois — mas escrita concorrente vira problema rápido.

Conjunto mínimo — e o que já existe

ENTIDADE	JÁ EXISTE?	O QUE FAZER
Project	CLIENTE	Promover a forma de <code>demoProjects()</code> para o servidor: <code>{id, name, description, aspect, createdAt, updatedAt}</code> . O id já é o segmento de caminho de <code>runtime/projects/</code> — mantenha a regex <code>^[A-Za-z0-9_-]{1,64}\$</code> e não invente um segundo identificador.
Scene	CLIENTE	Mover <code>createScene()</code> de <code>lib/storyboard.js</code> — a forma já está certa. Acrescentar só <code>projectId</code> . <code>SCENE_STATUS</code> permanece como está.
Asset	NÃO	A entidade nova de maior valor. <code>{id, projectId, kind:'image' 'video', jobId, filename, url, mimeType, bytes, width, height, durationSeconds, prompt, seed, modelId, derivedFromAssetId, status, createdAt}</code> . <code>derivedFromAssetId</code> é o que faz “anime essa imagem” funcionar. Substitui <code>listRegisteredVideos()</code> por consulta — mantendo a <code>allowlist</code> como defesa em profundidade.
Job	MEMÓRIA	Não reescrever. Persistir o recorte de <code>publicJob()</code> (<code>comfy/provider.js:820</code>) — o formato público já é o modelo, e é o que <code>og.get_job</code> devolve. Acrescentar <code>threadId</code> nulável.
AgentThread	NÃO	<code>{id, projectId, title, runtimeId:'hermes', status, createdAt, updatedAt}</code> .
AgentMessage	FORMA	A forma do cliente já serve: <code>{id, threadId, role:'user' 'agent' 'tool', text, toolCalls, assetIds[], createdAt}</code> . Note que <code>attachments:[id]</code> em <code>AgentScreen.jsx</code> vira <code>assetIds</code> — mudança de nome, não de conceito.

Deixe de fora do primeiro corte: Shot, Take, Production, Approval como entidade, memória do projeto. Nenhum é necessário para os dois slices, e cada um deles é mais fácil de acertar depois que Asset existir.

Arquivos que provavelmente mudarão

ARQUIVO	MUDANÇA	RISCO
<code>lib/server/comfy/config.js</code>	WORKFLOW_PATH/NODE_IDS/NODE_CLASSES passam de constantes globais a entradas por workflow.	ALTO
<code>lib/server/comfy/workflow.js</code>	<code>loadWorkflowTemplate()</code> e <code>validateWorkflow()</code> recebem um descritor. <code>patchWorkflow()</code> vira específico por workflow.	ALTO
<code>lib/server/comfy/storage.js</code>	Acrescentar <code>validateImageFilename()</code> , <code>imageDirFor()</code> , <code>publishImageFile()</code> , <code>imageUrlFor()</code> . Não tocar no que existe.	MÉDIO
<code>lib/server/comfy/status.js</code>	<code>findImageOutput()</code> ao lado de <code>findVideoOutput()</code> ; generalizar <code>jobIdFromFilename()</code> .	MÉDIO
<code>lib/server/comfy/provider.js</code>	862 linhas assumindo vídeo (<code>validarMp4</code> , <code>videoTempPath</code> , <code>publishVideoFile</code>). Extrair o esqueleto <code>submit</code> → <code>poll</code> → <code>finalize</code> para <code>lib/server/generation/</code> e parametrizar por tipo de mídia.	ALTO
<code>app/api/media/[...segments]/route.js</code>	Uma entrada nova no mapa TIPOS: <code>image</code> .	BAIXO
<code>components/navigation.js</code>	Uma linha em TABS (ou reescrever o rótulo de agente).	BAIXO
<code>components/StudioShell.jsx</code>	Uma entrada em SCREENS.	BAIXO
<code>components/screens/AgentScreen.jsx</code>	Trocar <code>lib/agentScript.js</code> por chamadas ao <code>AgentProvider</code> . A árvore de <code><Message></code> e os anexos permanecem.	MÉDIO
<code>components/StudioContext.jsx</code>	Quando o servidor virar dono, <code>conversation/generations/projects</code> deixam de hidratar do <code>localStorage</code> .	ALTO
<code>lib/models.js</code>	Modelo de imagem real com <code>providerId</code> apontando para o caminho novo.	BAIXO
<code>lib/providers/registry.js</code>	Registrar o <code>AgentProvider</code> .	BAIXO
<code>lib/server/logs/stages.js</code>	<code>CHANNELS.AGENT</code> e estágios do turno.	BAIXO
<code>lib/storage.js</code>	Chaves migradas saem de <code>KEYS</code> .	MÉDIO
<code>README.md</code>	Está errado hoje. Atualize junto com a primeira mudança real.	BAIXO

Arquivos que não deveriam ser alterados

Cada um destes resolve um problema real, difícil, já pago — vários carregam a cicatriz do bug que os originou documentada no cabeçalho. Estender é permitido; reescrever, não.

ARQUIVO	POR QUE É INTOCÁVEL
<code>lib/server/comfy/client.js</code>	Já é genérico. Toda troca é registrada em um só lugar; duplicar isso num “cliente do agente” quebra a rastreabilidade.
<code>lib/server/comfy/images.js</code>	Três garantias documentadas (magic bytes, nome derivado do jobId, bytes do usuário nunca reescritos). É puro e testado.
<code>lib/server/comfy/jobs.js</code>	73 linhas corretas. Acrescentar persistência ao redor, não dentro.
<code>lib/server/export/*</code>	Pipeline FFmpeg fechado e correto — allowlist, shell: false, rename atômico, limpeza de temporários com dupla condição. O agente não precisa entrar aqui no primeiro ano.
<code>lib/server/media/filmstrip.js</code>	Cache invalidado por hash do arquivo. Resolvido.
<code>lib/server/logs/store.js</code> · <code>persist.js</code> · <code>sanitize.js</code>	Retenção de erro na rotação, NDJSON com poda, redação de caminhos. Escreva <i>através</i> do <code>logger.js</code> .
<code>components/ui/RealVideoPlayer.jsx</code>	O cabeçalho descreve o bug real que o retry resolve. Use como está.
<code>app/api/export/*</code> · <code>app/api/filmstrip</code> · <code>app/api/logs</code>	Fora do escopo do agente.
<code>tests/comfy-recovery.test.mjs</code> · <code>finalize-robustness.test.mjs</code> · <code>export-*.test.mjs</code>	Regressões de falhas reais em produção. Se um destes quebrar, o refactor está errado — não o teste.
<code>runtime/projects/**</code>	Vídeos reais do usuário. Nenhum código novo escreve, move ou renomeia nada aqui.
<code>LICENSE</code> · <code>THIRD_PARTY_NOTICES.md</code>	Documento de conformidade.

Plano incremental

Onze passos. Cada um é testável isoladamente com `npm test`, e a ordem é escolhida para que **nenhum passo dependa do Hermes estar pronto** até o passo 6.

00 Colocar sob controle de versão

`git init` e um primeiro commit do estado atual. Não é cerimônia: os passos 3 e 8 mexem em `comfy/provider.js`, e sem histórico não há como voltar.

PRONTO QUANDO `git log` mostra o baseline e `npm test` passa nele.

01 Camada de domínio, sem agente nenhum

Criar `lib/server/domain/` com SQLite (`node:sqlite`) em `runtime/showrunner.db`: tabelas `projects`, `scenes`, `assets`. Portar `createScene()` de `lib/storyboard.js`. Backfill: varrer `runtime/projects/*/videos/` e inserir um `Asset` por MP4 existente — reutilizando `listRegisteredVideos()`.

PRONTO QUANDO um teste novo cria projeto e cena, e o backfill registra os 9 vídeos que já existem em disco sem tocar em nenhum arquivo.

02 Registry de workflows

Extrair de `comfy/config.js` um descritor por workflow: `{id, path, nodeIds, nodeClasses, requiredModels, kind: 'video' | 'image', patch(graph, params)}`. O MiniMax H3 vira a primeira entrada e **nada no comportamento muda**. O caminho passa a ser relativo a uma raiz configurável por env.

PRONTO QUANDO `tests/comfy-workflow.test.mjs` passa sem alteração e um segundo descritor fictício também valida.

03 Generalizar o pipeline para imagem

Extrair o esqueleto `submit→poll→finalize` de `comfy/provider.js` para `lib/server/generation/`, parametrizado por `mediaKind`. Acrescentar em `comfy/storage.js` as funções de imagem e em `status.js` o `findImageOutput()`. Validação: `ffprobe` para vídeo, dimensões para imagem.

PRONTO QUANDO os 21 testes existentes passam intactos e um novo prova que o pipeline publica um PNG em `runtime/projects/<id>/images/`.

04 Um workflow de imagem real, ainda sem agente

Acrescentar um workflow t2i ao registry (SDXL/Flux no ComfyUI é o caminho mais curto — Ideogram não tem uma linha de código aqui e é decisão à parte). Nova rota `app/api/generate/image` nos moldes exatos de `comfy/generate`, e o tipo `image` em `app/api/media/[...segments]/route.js`.

PRONTO QUANDO um `curl` gera uma imagem, ela aparece em `assets` e abre no navegador pela URL `/api/media/image/...`. Nenhum agente foi escrito ainda e metade do valor já está entregue.

05 Gateway com runtime falso

Criar `lib/server/agent/` completo — `AgentRuntimePort`, `gateway.js`, `threads.js`, `tools/registry.js` — com apenas o `EchoRuntimeAdapter`: dado o texto “dragão”, ele devolve uma `tool_call` fixa para `og.generate_image`. Tools do primeiro corte: `generate_image`, `get_job`.

PRONTO QUANDO um teste com o Echo executa o turno inteiro — mensagem → tool call → job → asset — sem rede e sem LLM.

06 HermesRuntimeAdapter

Segundo adaptador, atrás da mesma porta. Configuração por env (`HERMES_URL`, `HERMES_TOKEN`) no padrão de `COMFY_URL`. O catálogo de tools é publicado a partir do `registry.js` — uma fonte só. `testConnection()` nos moldes de `comfy/test`, com checagens nomeadas.

PRONTO QUANDO alternar `AGENT_RUNTIME=echo|hermes` troca o motor sem que nenhum outro arquivo mude. É aqui que o princípio nº 9 fica provado, não prometido.

07 A aba, e a imagem inline

Reescrever `AgentScreen.jsx` para falar com `/api/agent/*`. Manter `<Message>`, os anexos e o `ImageFrame`: passam a resolver `assetIds` em vez de ids do `localStorage`. Acompanhamento por polling primeiro — copiando o `acompanhar()` de `CinemaScreen.jsx`, com backoff.

PRONTO QUANDO “Crie uma imagem de um dragão.” produz a imagem inline na bolha do agente. Vertical slice 1 fechado.

08 og.generate_video com asset_id

Terceira tool. Recebe `asset_id`, resolve o `Asset` de imagem, envia os bytes a `uploadImage()` do `comfy/client.js` — o caminho i2v já funciona desde `comfy/images.js` e `attachFrames()`. Grava o vídeo com `derivedFromAssetId` apontando para a imagem.

PRONTO QUANDO “Agora anime essa imagem.” gera o MP4 pelo MiniMax H3 e o `RealVideoPlayer` o mostra inline. Vertical slice 2 fechado.

09

Fila e limites

Fila própria com concorrência 1 na frente do ComfyUI — resolve o R2 (`/interrupt` global) e o R7 de uma vez. Orçamento de tools por turno e timeout de job.

PRONTO QUANDO dois turnos simultâneos serializam e cancelar um não afeta o outro.

10

Streaming, cenas e aprovações

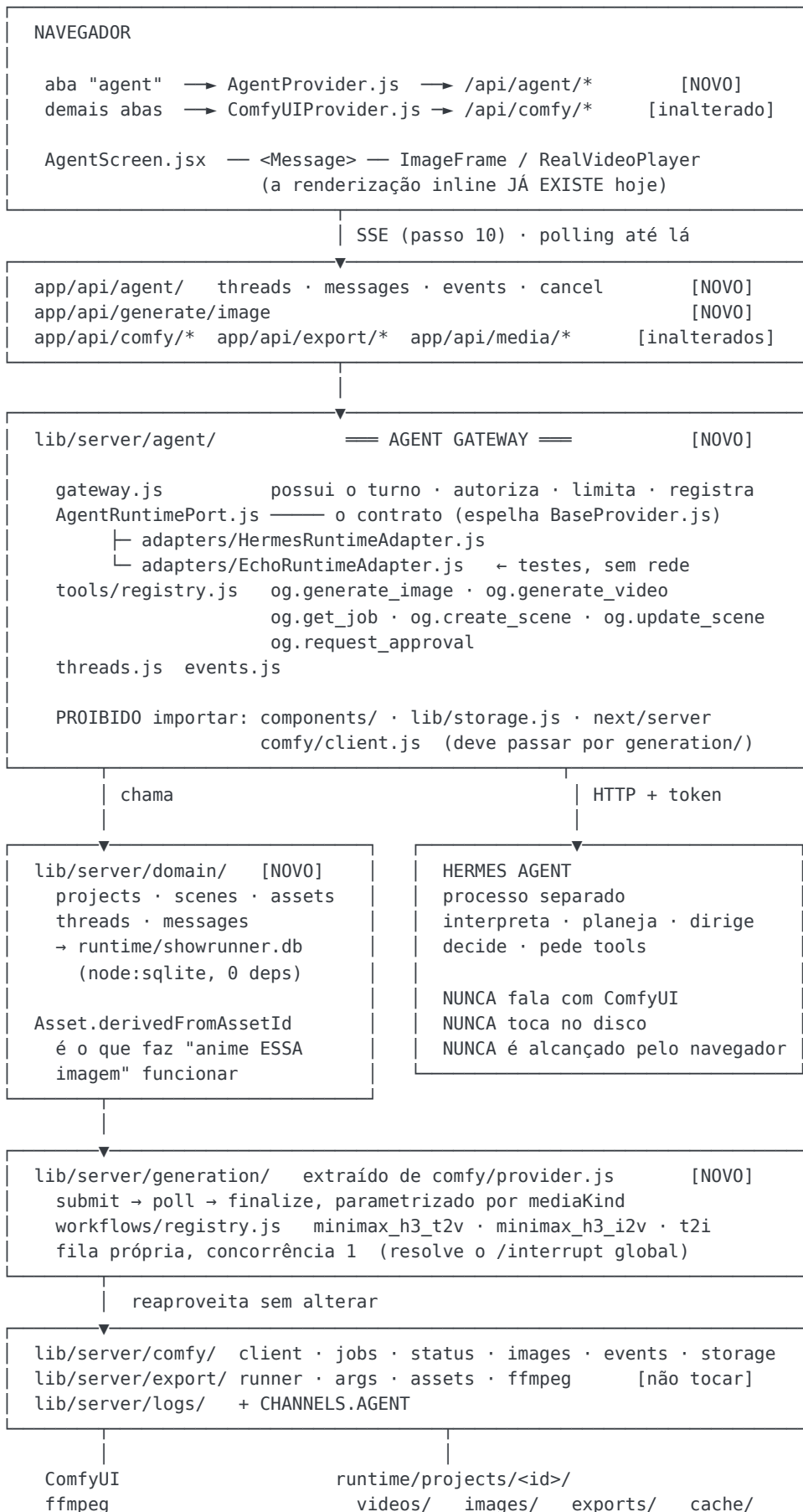
Só agora: SSE em `app/api/agent/events` (o padrão de cursor de `app/api/logs` já mostra o formato). Depois `og.create_scene`, `og.update_scene`, `og.request_approval` — que exigem o estado de aprovação migrado para o servidor.

PRONTO QUANDO o texto do agente aparece progressivamente e uma cena criada pelo agente aparece no Storyboard.

A ORDEM NÃO É ARBITRÁRIA

Os passos 1–4 entregam geração de imagem no Showrunner *sem nenhum agente*. Se o Hermes atrasar, mudar ou for trocado, esse trabalho continua valendo integralmente. Os passos 5–6 provam a porta com um adaptador falso antes de haver rede envolvida. Só o passo 7 acopla a UI. Nenhum passo exige que o anterior esteja perfeito — apenas que passe nos testes.

Diagrama da arquitetura proposta



Recomendação final

A arquitetura conceitual está certa e o repositório está mais perto dela do que parece — mas não pelo motivo esperado. A separação Frontend → Backend → Gateway → Hermes → tools → Jobs → ComfyUI encaixa com naturalidade porque toda a lógica já vive em `lib/server/` atrás de rotas finas, e porque `lib/providers/BaseProvider.js` já é, na prática, um `AgentRuntimePort` aplicado a outro problema. A equipe já pensa em portas.

O obstáculo real não é o Hermes — é que o domínio mora no navegador. O princípio nº 4 diz que o Showrunner é dono de projetos, cenas, assets, aprovações e memória. Hoje ele não é dono de nada disso: está tudo em `localStorage`, invisível para qualquer código de servidor. Um agente que rode no servidor e não consiga ler o próprio projeto não é um agente — é um gerador de prompts. *Comece pelo passo 1, não pelo Hermes.*

O segundo obstáculo é que geração de imagem não existe em nenhuma forma. Não é um provider desligado: é a ausência de workflow, de armazenamento, de tipo de mídia e de rota. Ideogram tem zero linhas de código. Os passos 2–4 entregam isso *sem escrever uma linha de agente* — e é aí que está o melhor retorno do projeto inteiro, porque continua valendo mesmo que Hermes seja substituído.

O que já está pronto e surpreende: a UI do chat com anexos inline de imagem e vídeo existe hoje, funcionando, em `AgentScreen.jsx`. O `RealVideoPlayer` existe. O `VideoFrame` já ramifica entre real e simulado. Quando o passo 7 chegar, a camada visual do vertical slice será a menor parte do trabalho — o que confirma que a fatia certa a atacar é o servidor.

Duas decisões a tomar antes da primeira linha: (1) o nome — o produto já se chama Showrunner, e chamar o agente de Showrunner cria ambiguidade permanente no código e na conversa com o usuário; (2) o modelo de imagem — “Ideogram” no catálogo é rótulo, não integração, e um workflow t2i no ComfyUI que já está de pé é caminho muito mais curto que um cliente de API novo. Escolher SDXL ou Flux para o slice 1 e trazer Ideogram depois custa menos e prova o pipeline antes.

Sobre o rigor existente: este código tem rename atômico, validação por `ffprobe` antes de publicar, magic bytes em vez de extensão, `allowlist` contra travessia, recuperação de jobs órfãos e testes que documentam falhas reais de produção. Isso é raro. A tentação ao integrar um agente é contorná-lo — um cliente HTTP próprio aqui, um caminho de arquivo direto ali. *Não contorne.* Se um handler de tool precisar de algo que a camada de baixo não oferece, estenda a camada de baixo. É o que preserva a única coisa que não se recompra depois.

Sobre este documento

NATUREZA	Auditoria arquitetural somente-leitura. Nenhum arquivo do repositório foi criado, alterado ou removido; nenhuma migração, dependência ou refactor foi executado.
REPOSITÓRIO	/media/douglas/SSD2/dev/open/showrunner-studio — package.json declara "name": "showrunner-studio", versão 0.1.0, licença MIT.
BASE ANALISADA	62 arquivos-fonte somando 15.071 linhas em app/, components/ e lib/, mais 21 arquivos de teste em tests/ e o conteúdo de runtime/.
DATA	2 de setembro de 2026
ESCOPO DAS SEÇÕES	As investigações A–F correspondem ao capítulo 1; G ao capítulo 6; H ao capítulo 7; I ao capítulo 10. Os doze capítulos correspondem ao formato de resposta solicitado.